

Actúa como asistente de programación para una práctica de Estructura de Datos en Python.

Contexto:

Ya tengo un proyecto desarrollado en Visual Studio Code con Python, programación orientada a objetos y una interfaz gráfica creada con Qt Designer. En el proyecto ya existe una clase `Queue` con los métodos básicos de una cola, un archivo principal y una clase que carga o controla la interfaz.

Objetivo:

Necesito integrar una simulación de cola de impresión. Cada trabajo debe contener usuario, documento, número de páginas y consecutivo. Los trabajos deben atenderse estrictamente en orden FIFO. También necesito que generes o ajustes la interfaz gráfica en Qt Designer mediante un archivo `.ui` que cumpla las especificaciones indicadas.

Instrucciones importantes:

1. No generes un proyecto nuevo desde cero.
2. No reemplaces ni reescribas por completo la clase `Queue` existente.
3. No uses una lista directamente como sustituto de `Queue`.
4. Respeta la organización actual de archivos, clases, nombres de métodos y estilo del proyecto.
5. Propón una clase `TrabajoImpresion` y una clase `GestorImpresion` que contenga una instancia de `Queue`.
6. Indica cambios mínimos y explica exactamente en qué archivo y en qué parte deben colocarse.
7. La interfaz debe permitir agregar trabajos, visualizar la cola, consultar el frente y procesar el siguiente trabajo.
8. Valida campos vacíos, páginas menores que 1 y el intento de procesar una cola vacía.
9. Explica cómo se aplican `enqueue`, `dequeue`, `front/peek`, `is_empty` y `size`, y verifica que estas operaciones se utilicen realmente.
10. Genera o ajusta un archivo de interfaz de Qt Designer con extensión `.ui`. No construyas toda la interfaz directamente con código Python, salvo la carga y conexión de eventos que ya utilice el proyecto.
11. La interfaz `.ui` debe incluir como mínimo:
 - Campos para capturar usuario, nombre del documento y número de páginas.
 - Botón Agregar a la cola.
 - Botón Imprimir siguiente.
 - Botón Consultar frente o un área visible que muestre el trabajo ubicado al frente.
 - Una tabla o lista para visualizar los trabajos pendientes en orden FIFO.
 - Una etiqueta con el total de trabajos pendientes.
 - Un área de mensajes para mostrar errores, confirmaciones y el trabajo procesado.

Entendido. Me enfocaré estrictamente en proporcionarte la integración exacta bajo la arquitectura que ya tienes armada, aplicando composición para no tocar tu clase `Queue` y estructurando la interfaz en formato `.ui`.

1. Explicación de Operaciones de la Cola (`Queue`)

Para cumplir con el comportamiento **FIFO** (First In, First Out), las operaciones se mapean así en nuestra simulación:

- `enqueue(item)`: Inserta un objeto `TrabajoImpresion` al final de la cola cuando el usuario presiona "Agregar".
- `dequeue()`: Remueve y retorna el trabajo al frente de la cola cuando se presiona "Imprimir siguiente".
- `front()` o `peek()`: Consulta el elemento que está al inicio de la cola sin removerlo, ideal para la sección "Consultar frente".

- `is_empty()`: Devuelve un booleano. Se usa obligatoriamente antes de hacer un `dequeue()` o `front()` para evitar errores si la cola está vacía.
 - `size()`: Devuelve la cantidad de elementos. Se usa para actualizar la etiqueta de "Total de trabajos pendientes".
-

2. Nuevas Clases del Modelo (`modelos.py` o sección de lógica)

Crea un archivo nuevo llamado `gestion_impresion.py` (o colócalo en tu archivo de modelos existente) con la lógica del negocio:

Python

```
# gestion_impresion.py

class TrabajoImpresion:
    def __init__(self, usuario: str, documento: str, paginas: int,
consecutivo: int):
        self.usuario = usuario
        self.documento = documento
        self.paginas = paginas
        self.consecutivo = consecutivo

    def __str__(self):
        return f"[{self.consecutivo}] {self.usuario} - {self.documento}
({self.paginas} págs.)"

class GestorImpresion:
    def __init__(self, queue_instance):
        # Recibe la instancia de la clase Queue existente (Composición)
        self.cola = queue_instance
        self._contador_consecutivo = 1

    def agregar_trabajo(self, usuario: str, documento: str, paginas: int)
-> tuple[bool, str]:
        # Validaciones requeridas
        if not usuario.strip() or not documento.strip():
            return False, "Error: El usuario y el documento no pueden
estar vacíos."
        if paginas < 1:
```

```

        return False, "Error: El número de páginas debe ser mayor o
igual a 1."

    # Crear e insertar
    nuevo_trabajo = TrabajoImpresion(usuario.strip(),
documento.strip(), paginas, self._contador_consecutivo)
    self.cola.enqueue(nuevo_trabajo) # Uso de enqueue
    self._contador_consecutivo += 1
    return True, f"Trabajo '{nuevo_trabajo.documento}' agregado con
éxito."

def procesar_siguiente(self) -> tuple[bool, str]:
    if self.cola.is_empty(): # Uso de is_empty
        return False, "Error: No hay trabajos pendientes en la cola
de impresión."

    trabajo_atendido = self.cola.dequeue() # Uso de dequeue
    return True, f"Imprimiendo: {trabajo_atendido}"

def obtener_frente(self) -> str:
    if self.cola.is_empty():
        return "Cola vacía"
    # Asumiendo que tu clase Queue implementa front() o peek()
    # Si se llama diferente (ej. peek), cámbialo aquí.
    trabajo_frente = self.cola.front()
    return str(trabajo_frente)

def obtener_total(self) -> int:
    return self.cola.size() # Uso de size

def listar_trabajos(self) -> list:
    # Para visualizar sin romper la encapsulación ni vaciar la cola,
    # accedemos temporalmente a la estructura interna de tu Queue.
    # Si tu Queue usa una lista interna llamada self.items, la leemos
ordenadamente:
    try:
        return [str(item) for item in self.cola.items]
    except AttributeError:

```

```
        # Ajusta 'items' al nombre del atributo/lista interna de tu
clase Queue

        return []
```

3. Integración en el Controlador de la Interfaz (`main.py` o controlador)

En tu clase encargada de manejar la ventana de la interfaz gráfica, realiza los siguientes cambios mínimos:

Importaciones y `__init__`

Python

```
# Importar el gestor y tu clase Queue existente
from queue_file import Queue # Reemplaza queue_file por el nombre real de
tu archivo
from gestion_impresion import GestorImpresion

# Dentro del __init__ de tu clase de interfaz (ej. MainWindow):
self.gestor = GestorImpresion(Queue()) # Pasamos una instancia limpia de
tu Queue

# Conexión de Eventos con los botones del archivo .ui
self.ui.btnAgregar.clicked.connect(self.ui_agregar_trabajo)
self.ui.btnImprimir.clicked.connect(self.ui_procesar_siguiente)
self.ui.btnConsultarFrente.clicked.connect(self.ui_consultar_frente)

# Inicializar componentes visuales
self.actualizar_pantalla()
```

Métodos del Controlador

Python

```
def ui_agregar_trabajo(self):
    usuario = self.ui.txtUsuario.text()
    documento = self.ui.txtDocumento.text()
    paginas = self.ui.spinPaginas.value() # O QLineEdit pasándolo a int

    exito, mensaje = self.gestor.agregar_trabajo(usuario, documento,
paginas)
```

```

self.ui.lblMensajes.setText(mensaje)

if exito:
    self.ui.txtUsuario.clear()
    self.ui.txtDocumento.clear()
    self.ui.spinPaginas.setValue(1)
    self.actualizar_pantalla()

def ui_procesar_siguiente(self):
    exito, mensaje = self.gestor.procesar_siguiente()
    self.ui.lblMensajes.setText(mensaje)
    self.actualizar_pantalla()

def ui_consultar_frente(self):
    frente = self.gestor.obtener_frente()
    self.ui.lblFrente.setText(f"Frente actual: {frente}")

def actualizar_pantalla(self):
    # 1. Actualizar total de pendientes
    self.ui.lblTotalPendientes.setText(f"Trabajos en cola: {self.gestor.obtener_total()}")

    # 2. Actualizar lista visual (QListWidget)
    self.ui.listTrabajos.clear()
    for trabajo_str in self.gestor.listar_trabajos():
        self.ui.listTrabajos.addItem(trabajo_str)

    # Opcional: Actualizar automáticamente el frente en un área visible
    self.ui_consultar_frente()

```

4. Archivo de Interfaz de Qt Designer (cola_impresion.ui)

Guarda este contenido en un archivo llamado `cola_impresion.ui`. Cuenta con los `objectName` exactos utilizados en el código superior:

```

XML
<?xml version="1.0" encoding="UTF-8"?>
<ui version="4.0">
    <class>ColaImpresionWidget</class>

```

```

<widget class="QWidget" name="ColaImpresionWidget">
  <property name="geometry">
    <rect>
      <x>0</x>
      <y>0</y>
      <width>500</width>
      <height>450</height>
    </rect>
  </property>
  <property name="windowTitle">
    <string>Simulación de Cola de Impresión</string>
  </property>
  <layout class="QVBoxLayout" name="verticalLayout">
    <!-- Campos de captura -->
    <layout class="QFormLayout" name="formLayout">
      <item row="0" column="0">
        <widget class="QLabel" name="label"><property
name="text"><string>Usuario:</string></property></widget>
        </item>
        <item row="0" column="1">
          <widget class="QLineEdit" name="txtUsuario"/>
        </item>
        <item row="1" column="0">
          <widget class="QLabel" name="label_2"><property
name="text"><string>Documento:</string></property></widget>
          </item>
          <item row="1" column="1">
            <widget class="QLineEdit" name="txtDocumento"/>
          </item>
          <item row="2" column="0">
            <widget class="QLabel" name="label_3"><property
name="text"><string>Páginas:</string></property></widget>
            </item>
            <item row="2" column="1">
              <widget class="QSpinBox" name="spinPaginas"><property
name="minimum"><number>0</number></property></property></widget>
              </item>
            </layout>

```

```

<!-- Botón Agregar -->
<widget class="QPushButton" name="btnAgregar">
    <property name="text"><string>Agregar a la cola</string></property>
</widget>

<line><property
name="orientation"><enum>Qt::Horizontal</enum></property></line>

<!-- Visualización de la cola -->
<widget class="QLabel" name="lblTotalPendientes">
    <property name="text"><string>Trabajos en cola: 0</string></property>
</widget>
<widget class="QListWidget" name="listTrabajos"/>

<!-- Área del Frente y Controles -->
<widget class="QLabel" name="lblFrente">
    <property name="text"><string>Frente actual: Cola
vacía</string></property>
</widget>
<layout class="QHBoxLayout" name="horizontalLayout">
    <widget class="QPushButton" name="btnConsultarFrente">
        <property name="text"><string>Consultar Frente</string></property>
    </widget>
    <widget class="QPushButton" name="btnImprimir">
        <property name="text"><string>Imprimir siguiente</string></property>
    </widget>
</layout>

<!-- Área de Mensajes -->
<widget class="QLabel" name="lblMensajes">
    <property name="styleSheet"><string>color: blue; font-weight:
bold;</string></property>
    <property name="text"><string>Área de
notificaciones</string></property>
    <property name="wordWrap"><bool>true</bool></property>
</widget>
</layout>
</widget>
<resources/>

```

```
<connections/>  
</ui>
```

Comence por darle el prompt con el contexto y le di mi clase queue para que me generara lo que le pedi despues de uwe me lo dio solo lo copie y ajuste y al final quedo el resultado esperado.